

Assignment #3

Analysis of Algorithms

Q1 The Floyd-Warshall algorithm is a dynamic programming approach used to find the shortest paths between all pairs of vertices in a weighted graph. Its central concept is to systematically improve the estimate of the shortest path between any two nodes i and j by considering an intermediate vertex k . By iterating through every vertex in the graph and treating it as a 'pivot' or stepping stone, the algorithm determines if going from $i \rightarrow k \rightarrow j$ is shorter than the current direct path from $i \rightarrow j$.

Formula:

$$A^k[i, j] = \min(A^{k-1}[i, j], A^{k-1}[i, k] + A^{k-1}[k, j])$$

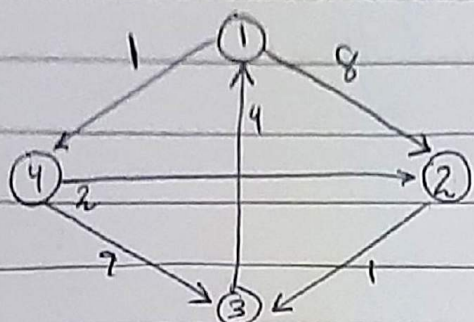
$A^{k-1}[i, j]$ = The shortest path from i to j using only vertices $\{1, 2, \dots, k-1\}$.

$A^{k-1}[i, k] + A^{k-1}[k, j]$ = The shortest path from i to j passing through the new intermediate vertex k .

A negative weight cycle occurs when the sum of edge weights in a cycle is less than zero. Floyd-Warshall detects this by checking the main diagonal of the distance matrix (the distance from a vertex to itself).

1. Initially, the distance $A[i, i]$ is set to zero.
2. If after the algorithm finishes, any diagonal element $A[i, i] < 0$, it proves that a path exists from vertex i back to itself with a total negative weight.
3. This indicates the presence of a negative weight cycle.

Q3



$$A^0 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 8 & \infty & 1 \\ 2 & \infty & 0 & 1 & \infty \\ 3 & 4 & \infty & 0 & \infty \\ 4 & \infty & 2 & 7 & 0 \end{array}$$

$$A^1 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 8 & \infty & 1 \\ 2 & \infty & 0 & 1 & \infty \\ 3 & 4 & \textcircled{12} & 0 & \textcircled{5} \\ 4 & \infty & 2 & 7 & 0 \end{array}$$

$$A^0[2,3] = A^0[2,1] + A^0[1,3]$$

$$1 < \infty + \infty$$

$$A^0[2,4] = A^0[2,1] + A^0[1,4]$$

$$\infty \approx \infty + 1$$

$$A^0[3,2] = A^0[3,1] + A^0[1,2]$$

$$\infty > 4 + 8$$

$$\infty > 12$$

$$A^0[3,4] = A^0[3,1] + A^0[1,4]$$

$$\infty > 4 + 1$$

$$\infty > 5$$

$$A^0[4,2] = A^0[4,1] + A^0[1,2]$$

$$2 < \infty + 8$$

$$A^0[4,3] = A^0[4,1] + A^0[1,3]$$

$$9 < \infty + \infty$$

$$A^2 = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 8 & 9 & 1 \\ 2 & \infty & 0 & 1 & \infty \\ 3 & 4 & 12 & 0 & 5 \\ 4 & \infty & 2 & 3 & 0 \end{array}$$

$$A^1[1,3] = A^1[1,2] + A^1[2,3]$$

$$\infty > 8 + 1$$

$$\infty > 9$$

$$A^1[1,4] = A^1[1,2] + A^1[2,4]$$

$$1 < 8 + \infty$$

$$A^1[3,1] = A^1[3,2] + A^1[2,1]$$

$$4 < 12 + \infty$$

$$A^1[3,4] = A^1[3,2] + A^1[2,4]$$

$$5 < 12 + \infty$$

$$A^1[4,1] = A^1[4,2] + A^1[2,1]$$

$$\infty \approx 2 + \infty$$

$$A^1[4,3] = A^1[4,2] + A^1[2,3]$$

$$9 > 2 + 1$$

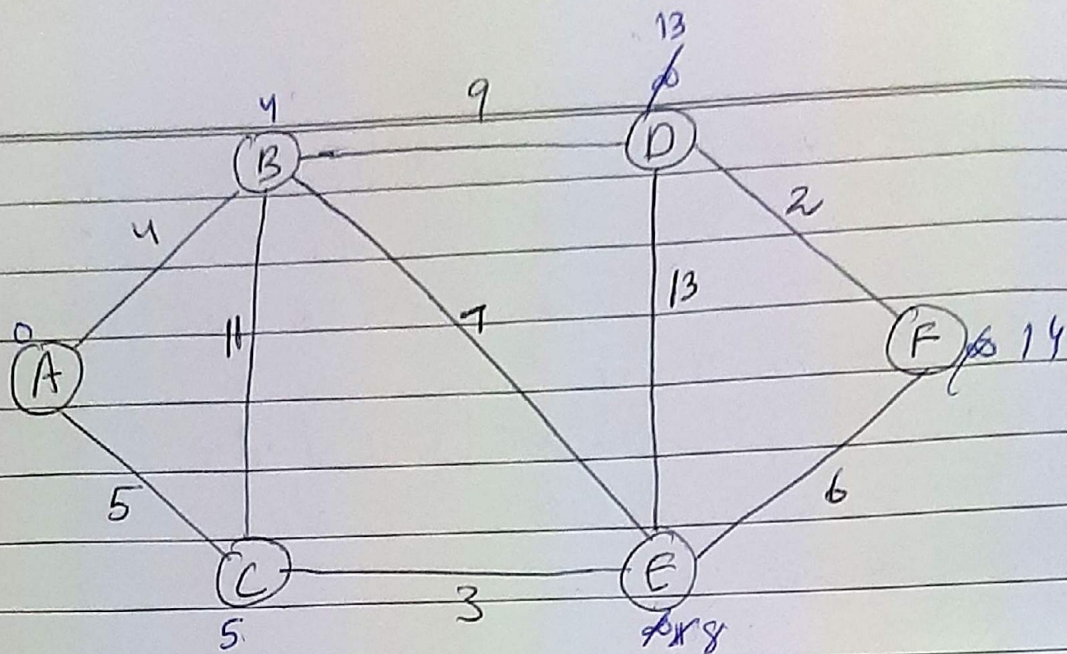
	1	2	3	4
A^2_2	0	8	9	1
	5	0	1	6
	4	12	0	5
	7	2	3	0

$A^2[1,2]$ $A^2[1,3] + A^2[3,2]$
 $8 < 9 + 12$
 $A^2[1,4]$ $A^2[1,3] + A^2[3,4]$
 $1 < 9 + 5$
 $A^2[2,1]$ $A^2[2,3] + A^2[3,1]$ $A^2[2,4]$ $A^2[2,3] + A^2[3,4]$
 $\infty > 1 + 4$ $\infty > 1 + 5$
 $\infty > 5$ $\infty > 6$
 $A^2[4,1]$ $A^2[4,3] + A^2[3,1]$ $A^2[4,2]$ $A^2[4,3] + A^2[3,2]$
 $\infty > 3 + 4$ $2 < 3 + 12$
 $\infty > 7$

	1	2	3	4
A_4	0	3	4	1
	5	0	1	6
	4	4 7	0	5
	7	2	3	0

$A^3[1,2]$ $A^3[1,4] + A^3[4,2]$
 $8 > 1 + 2$
 $8 > 3$
 $A^3[1,3]$ $A^3[1,4] + A^3[4,3]$
 $9 > 1 + 3$
 $5 < 6 + 7$ $9 > 4$
 $A^3[2,3]$ $A^3[2,4] + A^3[4,3]$ $A^3[3,1]$ $A^3[3,4] + A^3[4,1]$
 $1 < 6 + 3$ $4 < 5 + 7$
 $A^3[3,2]$ $A^3[3,4] + A^3[4,2]$
 $12 > 5 + 2$
 $12 > 7$

Q3



	Selected Vertices	B	C	D	E	F
$(5) < 15$	B	(4)	5	∞	∞	∞
$4+9 < \infty$	C	(4)	(5)	13	11	∞
$4+7 < \infty$	E	(4)	(5)	13	(8)	∞
$5+3 < 11$	D	(4)	(5)	(13)	(8)	14
$(13) < 13+8$	F	(4)	(5)	(13)	(8)	(14)
$8+6 < \infty$		4	5	13	8	14
$13+2 > (14)$						

Shortest distances from A

$$A \rightarrow B : 4$$

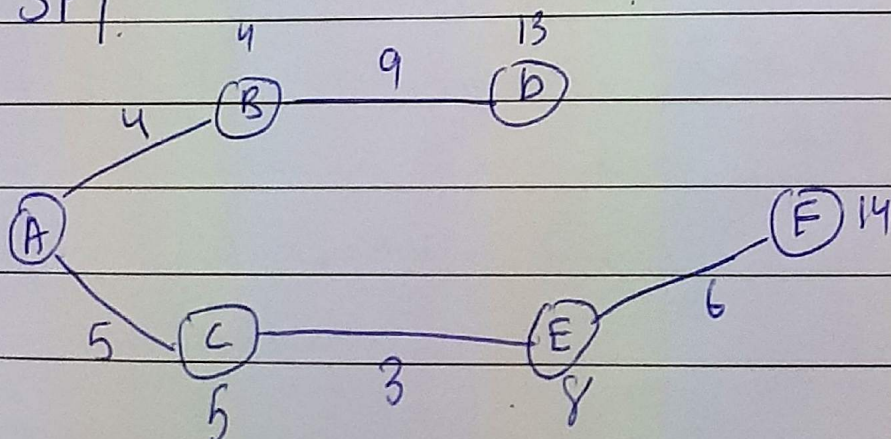
$$A \rightarrow C : 5$$

$$A \rightarrow D : 13 \text{ (via } A \rightarrow B \rightarrow D)$$

$$A \rightarrow E : 8 \text{ (via } A \rightarrow C \rightarrow E)$$

$$A \rightarrow F : 14 \text{ (via } A \rightarrow C \rightarrow E \rightarrow F)$$

SPT



Q4 Dijkstra is greedy because at every step, it makes the locally optimal choice. It looks at all vertices it hasn't visited yet and picks the one with the smallest known distance from the source. The algorithm assumes that because it has picked the absolute closest node, no other path (which would have to start by going through a further node) could possibly circle back and provide a shorter route.

Dijkstra fails with negative edge weights because once a vertex is visited, it assumes it has found the shortest possible path to that vertex and never checks it again. If a negative edge exists, a much longer path could suddenly become shorter later in the process. Since Dijkstra doesn't revisit nodes, it would miss this update, leading to an incorrect result.

The role of the Priority Queue is that it is basically the engine of the algorithm. Instead of searching the entire list of vertices to find the minimum distance (which takes $O(V)$ time), a min-heap allows us to extract the node with the smallest distance in $O(\log V)$ time. It also ensures that vertices are processed in strict order of their current shortest distance. This keeps the greedy property intact, as we always expand the frontier from the closest possible point.

Pseudocode:

Function `dijkstra(graph, source)`:

1. Create a distance array `d` & set all to infinity.
2. Set `d[source] = 0`
3. Initialize a Priority Queue, 'PQ' and insert `(0, source)`.

4. While PQ is not empty:

a. Pop the vertex v with the smallest distance from PQ

b. For each neighbor u of v :

i. Calculate $\text{new-dist} = d[v] + \text{weight}(v, u)$

Relaxation. $\Leftarrow$ ii. If $\text{new-dist} < d[u]$:

$d[u] = \text{new-dist}$

Push($\text{new-dist}, u$) to PQ

5. Return dist

Q5

Huffman Coding is a popular greedy algo used for lossless data compression. It assigns variable-length binary codes to characters based on their frequencies, more frequent characters get shorter codes, & less frequent characters get longer ones.

It ~~is~~ also ^{has} Prefix Free property meaning no code is a prefix of another like if A is 01, no other character can start with 01.

This ensures that the encoded bitstream is ~~a~~ unambiguous we can decode it from left to right without needing spaces b/w characters. It is optimal bc it minimizes the weighted external path length of the tree, which mathematically translates to the shortest possible average code length for a given set of frequencies.

The role of min-heap in Huffman ~~Tree~~ Coding is that it is used to build the Huffman tree from the bottom up:

1. Initialization that every character is placed into a min-heap as a leaf node, keyed by its frequency.

2. The algo repeatedly extracts the two nodes with the lowest frequencies from the heap and merge them.

3. A new internal node is created with a frequency equal to the sum of the two extracted nodes. These two nodes become the children of the new node.
4. We repeat the process. The new node is pushed back into the min-heap. This continues until only one node (the root) remains.

String: A A A A B C C

8 char. $\times$ 2 bits =

Fixed Encoding:

A = 00

B = 01

C = 10

Message Total bits = 8 char $\times$ 2 bits = 16 bits.

ASCII char bits: 3 Alphabets $\times$ 8 bits = 24 bits.

Huffman Coding: $\frac{\text{# of bits of code}}{\text{# of char}}$? $3 \times 2 = 6$ bits

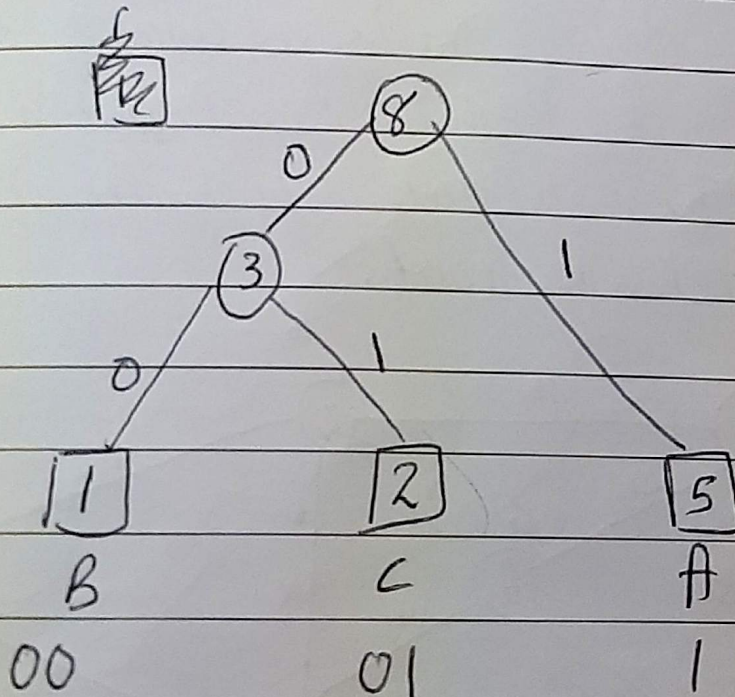
Total = 16 + 24 + 6 = 46 bits.

Huffman Coding:

A = 5

B = 1

C = 2



$\sum f_i \times l_i$

$1 \times 2 + 2 \times 2 + 5 \times 1 = 11$ bits.

So the bits reduced from 46 bits $\rightarrow$ 11 bits.

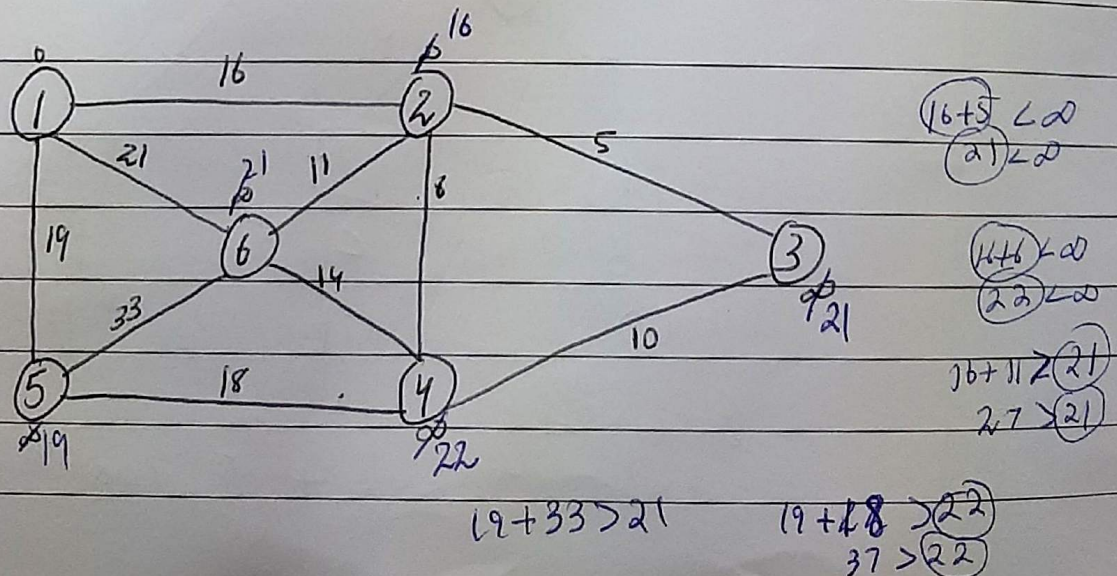
Pseudocode:

Algo Huffman(C):

~~1. Create a leaf node for every char.
2. Put all these nodes into a PQ sorted by freq.
3. While the PQ has more than 1 node in it.
a. Pop the node with the lowest frequency (call it left).
b. Pop the next node with the lowest frequency (call it right).
c. Create a new Parent node.
d. Set Parent's frequency = Left.freq + Right.freq.
e. Make left the left child & right the right child of Parent.
f. Push the new Parent node back into PQ.
4. The single node left in the PQ is the root of the tree.
5. Return the root.~~

1. Create a leaf node for every char.
2. Put all these nodes into a PQ sorted by freq.
3. While the PQ has more than 1 node in it.
 - a. Pop the node with the lowest frequency (call it left).
 - b. Pop the next node with the lowest frequency (call it right).
 - c. Create a new Parent node.
 - d. Set Parent's frequency = Left.freq + Right.freq.
 - e. Make left the left child & right the right child of Parent.
 - f. Push the new Parent node back into PQ.
4. The single node left in the PQ is the root of the tree.
5. Return the root.

Q6

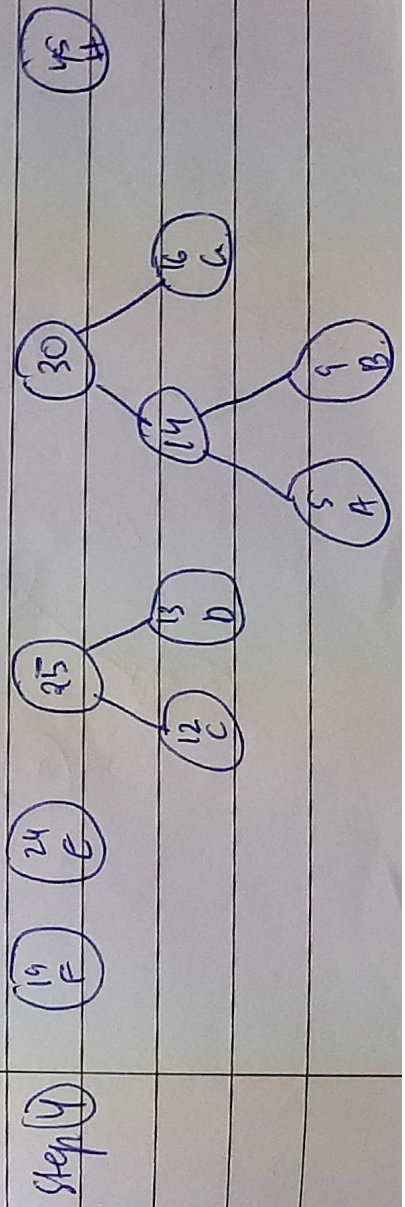
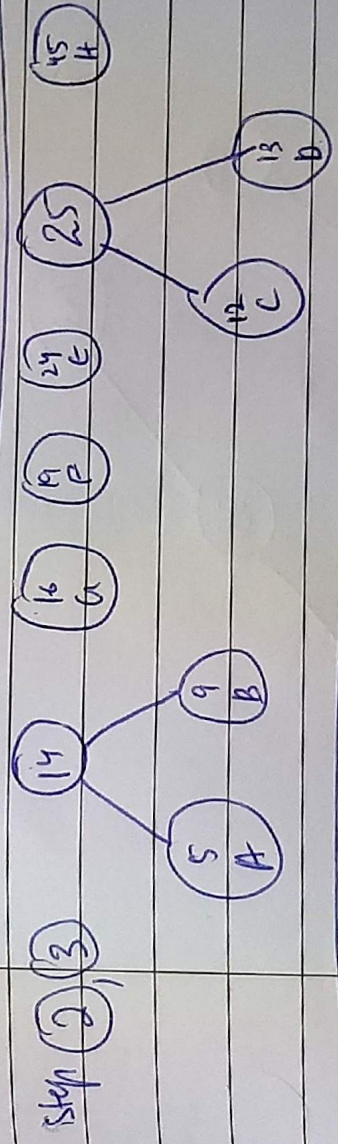
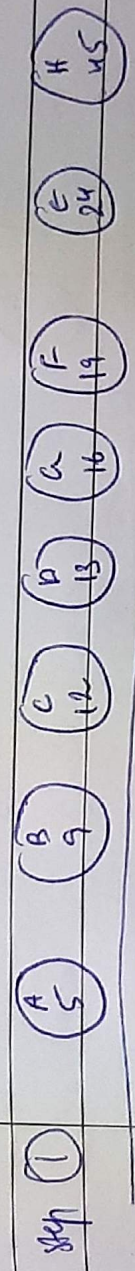


Selected Vertices	2	3	4	5	6
2	(16)	∞	∞	19	21
5	(16)	21	22	(19)	21
3	(16)	(21)	22	(19)	21
6	(16)	(21)	22	(19)	(21)
9	(16)	(21)	(22)	(19)	(21)
	16	21	22	19	21

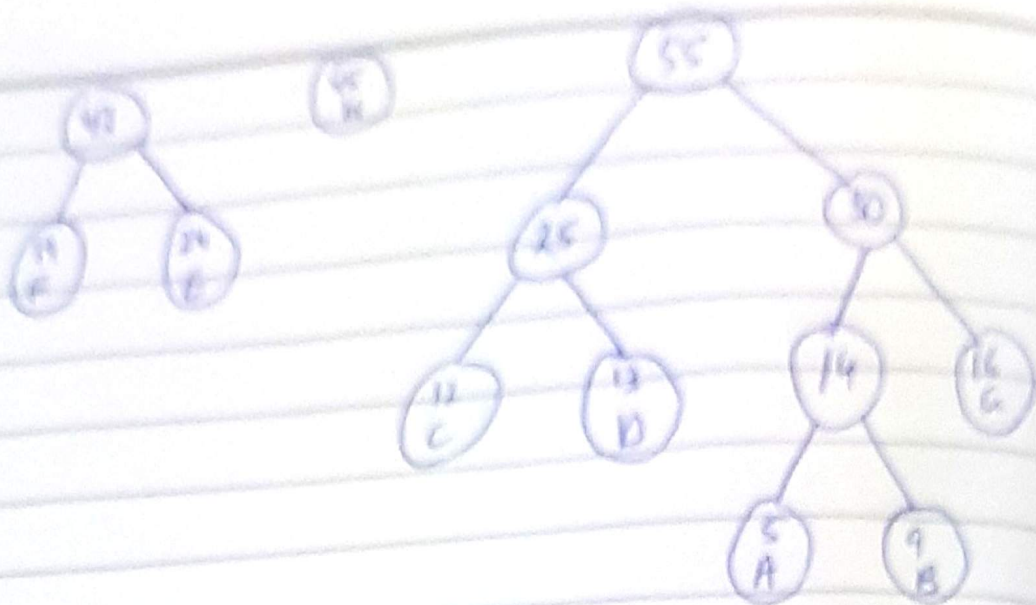
From vertex 1 to vertex 3 the shortest path is from
 $1 \rightarrow 2 \rightarrow 3$
 with distance = 21.

Q7

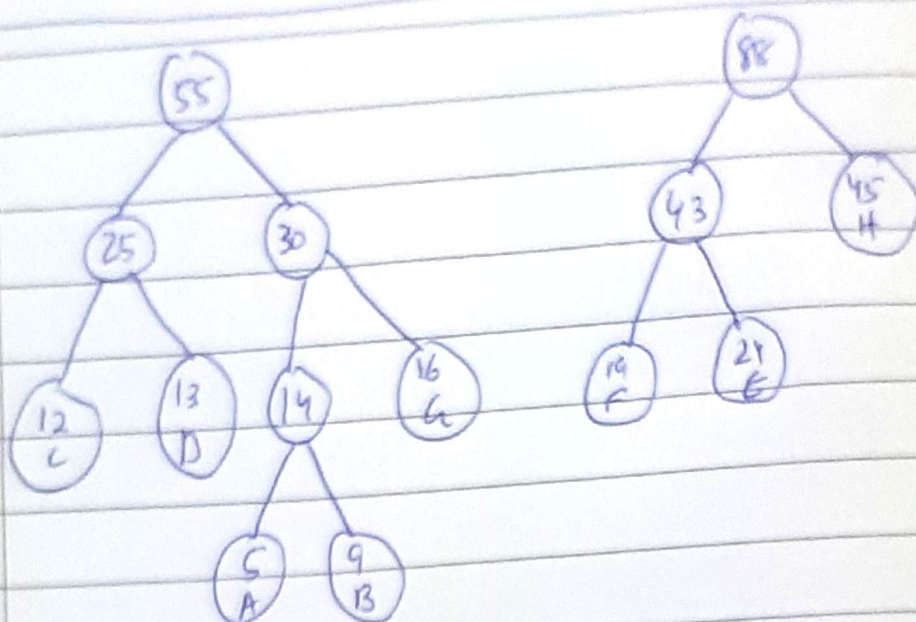
A	B	C	D	E	F	G	H
5	9	12	13	24	19	16	45



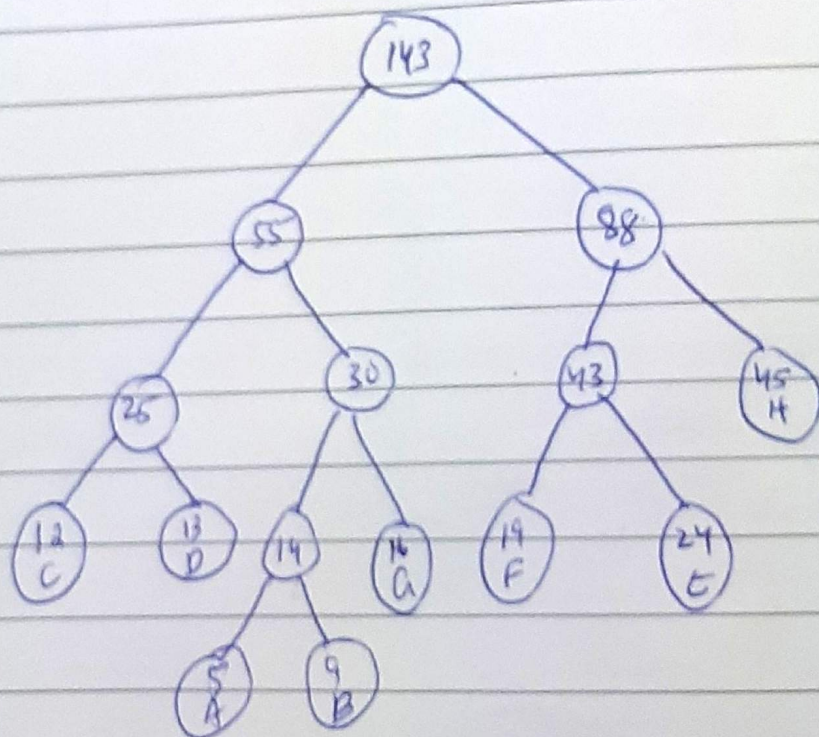
Step 56

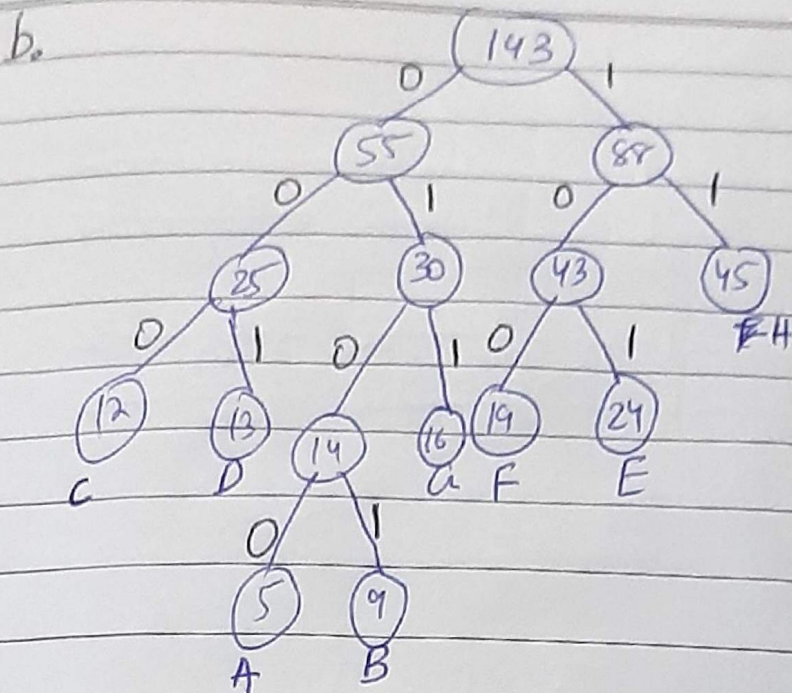


Step 7



Step 8





	C	D	A	B	G	F	E	H
Code	000	001	0100	0101	011	100	101	11
Freq	12	13	5	9	16	19	24	45

c. Total bits = $\sum f_i \times S_i$

$$= 12 \times 3 + 13 \times 3 + 5 \times 4 + 9 \times 4 + 16 \times 3 + 19 \times 3 + 24 \times 3 + 45 \times 2$$

$$= 398$$

Q8 a. Sort by profit

Job	Deadline.	Profit (\$)
J1	2	100
J5	3	60
J2	1	50
J4	1	20
J3	2	10

b. Slot allocation.

Max 3 deadlines.

0	1	2	3
0	1	2	3
0	1	2	3

0	1	2	3
0	1	2	3
0	1	2	3

Item rejected as deadline is 1 and slot 1 filled
Item 3 rejected as deadline is 2 and slot 2 filled

c. Total Jobs J_2, J_1, J_5

Total Profits = $50 + 100 + 60 = \$210$

Q1
In 0/1 Knapsack Problem, items are indivisible. You either take the whole item (1) or leave it (0). It requires Dynamic Programming.

In Fractional Knapsack Problem, Items can be broken into smaller pieces. You can take any fraction of an item. It is solved optimally by a Greedy Algorithm.

In the fractional version, a greedy strategy works perfectly. You simply sort items by their value-to-weight ratio (Value/Weight) and fill the bag with the most valuable per gram items until its full. If the last item doesn't fit, you can just cut it.

In 0/1 Knapsack, the greedy approach fails because choosing the item with the highest ratio might leave a small, awkward amount of empty space that cannot be

filled. A different combination of lower-ratio items might fill the bag more efficiently and yield a higher total value.

We solve 0/1 Knapsack with Dynamic Programming. Since we can't be greedy, we must explore all possible combinations. We use a 2-D Table (often called $dp[i][w]$) to store the results of subproblems.

The Table structure is $row(i)$: Represent the items available.
 $columns(w)$: Represent all possible weight capacities from 0 up to the maximum capacity W .

$$dp[i][w] = \max(dp[i-1][w], value_i + dp[i-1][w - weight_i])$$

For every cell in the table, the algo makes a choice:

1. Exclude the item: The value remains the same as the best value found for the previous items at that same weight.

2. Include the item: Add the current item's value to the best value found for the remaining weight capacity.

... and it assumes that once

Q10

backpack preventing a full load.

Profit	Weight	Items	0kg	1kg	2kg	3kg	4kg	5kg	6kg	7kg	8kg
40	2	0	0	0	0	0	0	0	0	0	0
50	3	1	0	0	40	40	40	40	40	40	40
60	4	2	0	0	40	50	50	90	90	90	90
70	5	3	0	0	40	50	60	100	110	110	110
		4	0	0	40	50	60	70	70	110	120

Item 1: 40, 2 $\Rightarrow$ At 2kg $\Rightarrow$ 40

Item 2: 50, 3 $\Rightarrow$ At 2+3=5kg $\Rightarrow$ 40+50=90

Item 3: 60, 4 $\Rightarrow$ At 4+2=6kg $\Rightarrow$ 60+40=100

Item 4: 70, 5 $\Rightarrow$ At 5+2=7kg $\Rightarrow$ 70+40=110, At 5+3=8kg $\Rightarrow$ 70+50=120
At 5+4=9kg reject X

Items chosen

x_1	x_2	x_3	x_4
0	1	0	1
			120-70=50

50-50=0

Weight = $0 \times 2 + 3 \times 1 + 0 \times 4 + 1 \times 5 = 8\text{kg}$ ✓

Profit = $0 \times 40 + 1 \times 50 + 0 \times 60 + 1 \times 70 = 120$ ✓

remains the same as the best value found for the previous items at that same weight.

2. Include the item: Add the current item's value to the best value found for the remaining weight capacity.

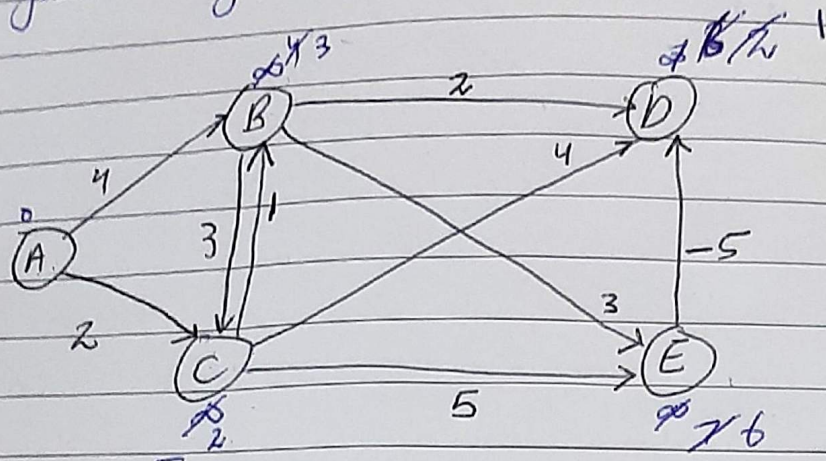


Dijkstra's algo is a greedy algo and it assumes that once a vertex is visited and its shortest distance is finalized, no other path can possibly be shorter.

In the graph of the assignment 3:

Dijkstra would likely reach vertex D from B with a distance of $6(4+2)$. And it would then mark D visited and finalize its distance. However, there is a path through C that is shorter: $A \rightarrow C \rightarrow D$, with cost 2, but because it never re-evaluates a finalized node, it

would completely miss this shorter path created by the negative weight edge.



5 vertices

$5-1 = 4$ time Relaxation is to be done.

EdgeList

(A,B) (A,C) (B,C) (B,D) (B,E) (C,B) (C,D) (C,E) (E,D)

	1st	2nd	3rd	Final	Path
A	0	0	0	0	Source
B	3	3		3	A → C → B
C	2	2		2	A → C
D	2	1		1	A → C → B → E → D
E	7	6		6	A → C → B → E

The shortest distance from A to D = 1

Q12

It is classified as NP Hard because it belongs to a category of combinatorial optimization problems for which no polynomial-time solution is known to exist. As the no. of cities increases, the search space also grows ~~fractionally~~ ^{factorially}, specifically at a rate of $\frac{(n-1)!}{2}$, making a ^{naive} force search mathematically im-

-possible for even small moderate sized datasets. ~~Because~~
it is NP Hard.

The Held-Karp algo uses dynamic programming to solve TSP more efficiently than brute force. Instead of looking at every possible permutation, it breaks the problem into sub-problems. Held Karp uses a memorization trick:

1. Breaks the big problems into smaller (like big trips to small)
2. Remembers the shortest way to visit a subset of cities,
3. If it needs that info again, it just looks it up in a table instead of recalculating it.

Time complexities:

Brute force: $O(n!)$ - Impossible if lots of values.

Held-Karp: $O(n^2 2^n)$ - Feasible.

In DP, a bitmask is a compact way of to represent a set of items or a collection of boolean states using a single integer. Each bit (0 or 1) at position i indicates whether the i th element is excluded or included in the current state.

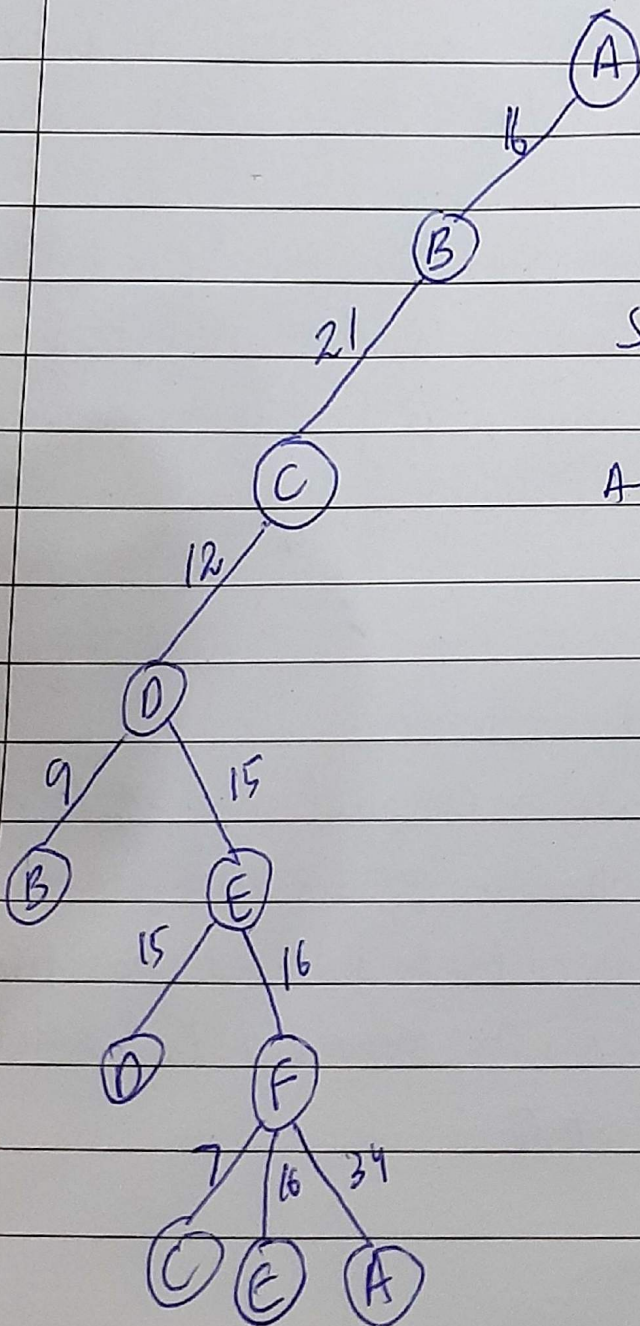
(a completed visit)

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow A$

Q13

1. Yes, a complete Hamiltonian cycle exists because there is a single directed path that visits every node exactly once, forming a ^{closed} loop.
2. Starting & ending at Node A.

	A	B	C	D	E	F
A	0	16	∞	∞	∞	∞
B	∞	0	21	∞	∞	∞
C	∞	∞	0	12	∞	∞
D	∞	9	∞	0	15	∞
E	∞	∞	∞	15	0	16
F	34	∞	7	∞	16	0



Total cost from A to A =

$$16 + 21 + 12 + 15 + 16 + 34 = 114$$

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow A$.

3. Since a cycle does exist, There is no functional bottleneck preventing a full loan.

Q10

		Items	0kg	1kg	2kg	3kg	4kg	5kg	6kg	7kg
Profit	Weight	0	0	0	0	0	0	0	0	0
40	2	1	0	0	40	40	40	40	40	40
50	3	2	0	0	40	50	50	90	90	90